

Command Reference

Cryptographic Cipher Analyzer - every command, option and example

Version 1.0.0 | 21 September 2026 | Run from the crypto_cipher_analyzer folder

EDUCATIONAL USE ONLY

Caesar and Vigenere are classical, broken ciphers. Use this tool to learn and to demonstrate cryptanalysis - never to protect real passwords or confidential data.

1. Quick start

```
cd crypto_cipher_analyzer          # all commands are run from here
python main.py                    # interactive menu (no arguments)
python main.py --help              # list every command
python main.py --version           # print the version

# the four commands people use first
python main.py caesar-encrypt --text "HELLO WORLD" --shift 3
python main.py caesar-decrypt --text "KHOOR ZRUOG" --shift 3
python main.py vigenere-encrypt --text "ATTACKATDAWN" --key LEMON
python main.py crack-caesar --input sample_data/caesar_sample.txt
```

Quote text that contains spaces. On Linux or macOS use python3 instead of python if the python command is not mapped.

2. How every command is written

```
python main.py [GLOBAL OPTIONS] <COMMAND> [OPTIONS]

GLOBAL OPTIONS  --help | --version | --no-color
COMMAND         one of the 13 commands listed in section 3 (omit it for the menu)
OPTIONS        --text, --input, --output, --shift, --key, --report, ...
```

Order matters for the global options

Global options must come BEFORE the command:

```
python main.py --no-color crack-caesar --text "KHOOR ZRUOG" correct
```

```
python main.py crack-caesar --no-color --text "KHOOR ZRUOG" error: unrecognized arguments
```

Anything after the command name belongs to that command.

3. Command index

Command	What it does	Reads its input from
interactive	Opens the numbered menu (default when no command is given).	- (asks interactively)
caesar-encrypt	Caesar encryption with a fixed shift.	--text or --input
caesar-decrypt	Caesar decryption with the same shift.	--text or --input
vigenere-encrypt	Vigenere encryption with a letters-only keyword.	--text or --input
vigenere-decrypt	Vigenere decryption with the same keyword.	--text or --input

Command	What it does	Reads its input from
frequency	Letter counts, percentages, English comparison, IC.	--text or --input
crack-caesar	Tries all 26 shifts and ranks the plaintexts.	--text or --input
analyze-vigenere	IC + Kasiski + per-column key recovery.	--text or --input
ic	Index of Coincidence, plus the average IC per key length.	--text or --input
kasiski	Repeated sequences, distances and factor ranking.	--text or --input
security	Strength ratings and known attacks for both ciphers.	- (no input)
file-caesar	Encrypts or decrypts a whole file with Caesar.	--input (required)
file-vigenere	Encrypts or decrypts a whole file with Vigenere.	--input (required)

4. Encryption and decryption commands

4.1 Caesar cipher

```
python main.py caesar-encrypt --text "TEXT" --shift N [--report]
python main.py caesar-decrypt --text "TEXT" --shift N [--report]

# examples
python main.py caesar-encrypt --text "HELLO WORLD" --shift 3      -> KHOOR ZRUOG
python main.py caesar-decrypt --text "KHOOR ZRUOG" --shift 3      -> HELLO WORLD
python main.py caesar-encrypt --text "Meet at dawn!" --shift -3    -> negative shifts allowed
python main.py caesar-encrypt --input notes.txt --shift 30         -> 30 is normalised to 4
python main.py caesar-encrypt --text "HELLO" --shift 3 --report    -> also saves a report
```

- The shift may be any whole number: negative values and values above 26 are reduced modulo 26 automatically.
- Upper case, lower case, spaces, punctuation, digits and newlines are all preserved.
- Missing or non-numeric shifts produce a friendly error, never a traceback.

4.2 Vigenere cipher

```
python main.py vigenere-encrypt --text "TEXT" --key WORD [--report]
python main.py vigenere-decrypt --text "TEXT" --key WORD [--report]

# examples
python main.py vigenere-encrypt --text "ATTACKATDAWN" --key LEMON  -> LXFOPVEFRNHR
python main.py vigenere-decrypt --text "LXFOPVEFRNHR" --key LEMON  -> ATTACKATDAWN
python main.py vigenere-encrypt --text "attack at dawn" --key lemon -> lxfopv ef rnhr
python main.py vigenere-encrypt --input message.txt --key SECRET
```

- The keyword must contain letters only (A-Z). 'LE MON', 'LEMON4' and 'LE-MON' are rejected with a message naming the offending characters.
- Spaces, punctuation and digits are copied through and do not consume a key letter, so 'ATTACK AT DAWN' stays aligned with 'ATTACKATDAWN'.

5. Cryptanalysis commands

5.1 frequency - letter frequency analysis

```
python main.py frequency --text "CIPHERTEXT" [--report]
python main.py frequency --input sample_data/caesar_sample.txt
```

Prints: total and alphabetic character counts, a count/percentage table for all 26 letters, the most and least common letters, a side-by-side comparison with standard English frequencies, and the Index of Coincidence as supporting evidence.

5.2 crack-caesar - break Caesar without the shift

```
python main.py crack-caesar --text "KH00R ZRU0G" [--top 5] [--report]
python main.py crack-caesar --input sample_data/caesar_sample.txt
```

Output: the ciphertext, a table of ranked candidates (rank, shift, score, plaintext), the best candidate with its estimated shift, and a note that the result is statistical.

Example result: KH00R ZRU0G -> rank 1 shift 3 score 40.81 -> HELLO WORLD

Limits of automatic cracking

The ranking scores how much each candidate looks like English. Very short messages, names, technical terms and non-English text can rank the wrong shift first, so compare the top few candidates instead of trusting row 1 blindly. Use --top N to see more.

5.3 ic - Index of Coincidence

```
python main.py ic --text "CIPHERTEXT" [--max-key-length 12] [--report]
python main.py ic --input sample_data/vigenere_sample.txt
```

Prints $IC = \frac{\sum(f_i(f_i-1))}{N(N-1)}$ with the English (0.0667) and random (0.0385) references, a plain-language interpretation, and the average IC of the columns for every assumed key length. The key length whose columns score highest is the best candidate - evidence, not proof.

5.4 kasiski - repeated sequence examination

```
python main.py kasiski --text "CIPHERTEXT" [--min-length 3] [--max-length 5] \
                        [--max-key-length 20] [--report]
```

Prints every repeated sequence of 3-5 letters with its positions, the distances between the repeats and the shared factors, then ranks candidate key lengths by how much evidence supports them.

- Every divisor of the true key length also appears in the ranking (a distance that is a multiple of 6 is automatically a multiple of 3), so treat the list as candidates.
- Use --min-length 4 to reduce noise on long texts, or --min-length 3 for short ones.

5.5 analyze-vigenere - full Vigenere cryptanalysis

```
python main.py analyze-vigenere --text "CIPHERTEXT" [--max-key-length 20] [--top 5] [--report]
python main.py analyze-vigenere --input sample_data/vigenere_sample.txt

# skip the estimation when you already know the key length
python main.py analyze-vigenere --input sample_data/vigenere_sample.txt --key-length 5

Workflow it performs automatically:
1. normalise the ciphertext          5. suggest the letters for each key position
2. measure the overall IC           6. build candidate keys and decrypt with each
3. run Kasiski examination          7. score the plaintexts and show the best guess
4. rank candidate key lengths

The shipped sample really is LEMON-encrypted, so this command recovers LEMON.
```

Every result is labelled honestly

'Estimated', 'candidate' and 'likely' are statistical hypotheses: only a readable decryption confirms them. If the output is gibberish, the key length was probably wrong: try the next candidate length from the table.

6. Security assessment command

```
python main.py security                    # both ciphers
python main.py security --cipher caesar    # Caesar only
python main.py security --cipher vigenere --report # Vigenere only, saved as a report

Caesar -> Security rating: VERY WEAK          (25 effective keys, brute force)
Vigenere -> Security rating: WEAK / EDUCATIONAL ONLY (Kasiski, IC, column frequency)
```

7. File encryption and decryption

```
python main.py file-caesar --input FILE [--output FILE] --shift N [--decrypt] [--overwrite]
python main.py file-vigenere --input FILE [--output FILE] --key WORD [--decrypt] [--overwrite]

# encrypt a file: writes notes_out.txt, notes.txt is left untouched
python main.py file-caesar --input notes.txt --shift 3

# decrypt into an explicit file
python main.py file-caesar --input notes_out.txt --output restored.txt --shift 3 --decrypt

# Vigenere round trip
python main.py file-vigenere --input notes.txt --output notes_enc.txt --key LEMON
python main.py file-vigenere --input notes_enc.txt --output notes_dec.txt --key LEMON --decrypt

# replace an existing output file on purpose
python main.py file-caesar --input notes.txt --output notes_out.txt --shift 3 --overwrite

# encrypt the shipped sample
python main.py file-caesar --input sample_data/caesar_sample.txt --shift 3
```

- --input is required and must exist; --output defaults to <name>_out.<extension>.
- An existing output file is never replaced unless you pass --overwrite (the interactive menu asks first).
- Writing back onto the input file itself is refused unless --overwrite is given.

8. Interactive menu commands

```
python main.py          # no arguments starts the menu

[1] Caesar Cipher      [6] File Encryption / Decryption
[2] Vigenere Cipher    [7] Security Analysis
[3] Frequency Analysis [8] Help
[4] Break Caesar Cipher [9] Exit
[5] Analyze Vigenere Cipher

Answers the menu understands:
  1-9 or 1-5   pick a menu entry (up to 5 options in submenus)
  b or back    go back one step
  y / n        answer a yes/no question (for example saving a report)
  Enter        accept the default shown in [brackets]
  Ctrl+C       cancel the current operation
```

9. Option reference

Option	Commands	Meaning
--text "..."	all text commands	Input text; quote it so spaces survive.
--input FILE	all text and file commands	Read the input from a file (required for file-caesar / file-vigenere).
--output FILE	file-caesar, file-vigenere	Destination file; default <name>_out.<ext>.
--shift N	caesar-encrypt, caesar-decrypt, file-caesar	Caesar shift; any integer, normalised modulo 26.
--key WORD	vigenere-encrypt, vigenere-decrypt, file-vigenere	Letters-only keyword, for example LEMON.
--decrypt	file-caesar, file-vigenere	Decrypt instead of encrypt.
--overwrite	file-caesar, file-vigenere	Allow replacing an existing output file.
--report	every command that prints a result	Save reports/analysis_YYYYMMDD_HHMMSS.txt without asking.
--top N	crack-caesar, analyze-vigenere (5)	How many candidates to display.
--max-key-length N	analyze-vigenere (20), ic (12), kasiski (20)	Largest key length to consider.
--key-length N	analyze-vigenere	Use this key length instead of estimating it.
--min-length / --max-length	kasiski (3 / 5)	Shortest and longest repeated sequence to search for.
--cipher {caesar,vigenere}	security	Limit the assessment to one cipher.
--no-color	global (before the command)	Disable ANSI colours for this run.
--help / --version	global (before the command)	Show help or the version and exit.

10. Copy-paste cookbook

I want to ...	Run this
Encrypt a short message with a shift	python main.py caesar-encrypt --text "HELLO WORLD" --shift 3
Decrypt a message when I know the shift	python main.py caesar-decrypt --text "KH00R ZRU0G" --shift 3

I want to ...	Run this
Find the shift when I do not know it	<code>python main.py crack-caesar --text "KH00R ZRU0G"</code>
Encrypt / decrypt with a keyword	<code>python main.py vigenere-encrypt --text "ATTACKATDAWN" --key LEMON</code>
Check whether a text looks monoalphabetic or polyalphabetic	<code>python main.py frequency --input sample_data/caesar_sample.txt</code>
Measure the Index of Coincidence and test key lengths	<code>python main.py ic --input sample_data/vigenere_sample.txt --max-key-length 12</code>
Look for repeated sequences and candidate key lengths	<code>python main.py kasiski --input sample_data/vigenere_sample.txt</code>
Recover an unknown Vigenere keyword	<code>python main.py analyze-vigenere --input sample_data/vigenere_sample.txt</code>
Force a known key length during recovery	<code>python main.py analyze-vigenere --input sample_data/vigenere_sample.txt --key-length 5</code>
Justify why the ciphers are weak (for a report or viva)	<code>python main.py security --report</code>
Encrypt an entire file without risking the original	<code>python main.py file-caesar --input notes.txt --shift 3</code>
Decrypt that file back	<code>python main.py file-caesar --input notes_out.txt --output restored.txt --shift 3 --decrypt</code>
Save any analysis as timestamped evidence	<code>python main.py frequency --input sample_data/caesar_sample.txt --report</code>
Regenerate this command sheet and the full guide	<code>python tools/generate_commands_pdf.py</code>

11. Where the output goes

- Screen: every command prints to standard output; results are never sent anywhere else.
- Reports: `--report` writes `reports/analysis_YYYYMMDD_HHMMSS.txt` inside the project folder, containing the date, cipher, operation, input statistics, analysis, candidates, security assessment and the educational note.
- Files: file commands write only to the path you name (or `<name>_out.<ext>`); they never modify the input file unless you explicitly overwrite it.
- Nothing is sent over the network: the tool is fully offline.

```
ls reports/                # list generated reports
cat reports/analysis_20260915_184500.txt  # read one (Linux/macOS)
type reports\analysis_20260915_184500.txt  # read one (Windows)
```

12. Test and maintenance commands

```
cd crypto_cipher_analyzer
python -m unittest discover tests          # run all 163 tests
python -m unittest discover tests -v       # verbose, one line per test
python -m unittest tests.test_caesar      # a single module
python -m unittest tests.test_cli -v      # the CLI / integration tests

python main.py --version                   # print the version
python -m compileall -q .                 # syntax-check every module
python tools/generate_commands_pdf.py     # rebuild this command sheet (reportlab needed)
```

13. Troubleshooting: message -> fix

Message printed	Cause and fix
[ERROR] Provide the input with --text "... " or --input <file>.	The command needs text. Add --text "YOUR TEXT" or --input FILE.
[ERROR] Provide a shift, for example --shift 3.	A Caesar command was run without --shift. Add --shift N.
[ERROR] Provide a keyword, for example --key LEMON.	A Vigenere command was run without --key. Add --key WORD.
[ERROR] 'banana' is not a valid shift. Use a whole number.	--shift must be a whole number; use 3, -3 or 30.
[ERROR] The keyword must contain letters only (A-Z). Remove: 4	The key has non-letter characters; remove them (spaces and digits are not allowed).
[ERROR] Input file not found: notes.txt	Wrong path or wrong folder: run commands from crypto_cipher_analyzer.
[ERROR] The file 'X' is empty; nothing to process.	The input file has no content: add text or pick another file.
[ERROR] Output file already exists: X. Choose another output path or enable overwrite...	Add --overwrite, or pass a different --output path.
[ERROR] The ciphertext is shorter than the requested key length; choose a smaller length.	Lower --key-length so every key position has letters to analyse.
[ERROR] At least eight letters are required for meaningful Vigenere analysis.	The text is too short for cryptanalysis: supply a longer ciphertext.
[ERROR] No letters were found in the input; nothing to analyse.	The input contains no A-Z letters (digits, punctuation or empty text).
[ERROR] unrecognized arguments: --no-color	Global options go before the command: python main.py --no-color <COMMAND> ...
[ERROR] 'banana' is not a valid selection. Choose one of: 1, 2, 3	Interactive mode: enter one of the listed numbers, or b to go back.
[ERROR] Unsupported operation: X (exit code 2)	Unknown command: run python main.py --help for the full list.
Command not found: python	Use python3 on Linux/macOS, or the py launcher on Windows: py main.py

No tracebacks, by design

Every invalid input, missing file and permission problem is caught and reported as a clear message. If you ever see a Python traceback, copy the command you ran - it is a bug.

14. Exit codes (for scripts)

Code	Meaning
0	Success - including --help and --version.
1	Handled error: invalid input, missing file, rejected operation.

Code	Meaning
2	Command-line usage error: unknown command or unrecognized arguments.
130	Cancelled by the user (Ctrl+C).

```
# use the exit code in a script
python main.py caesar-encrypt --text "HELLO" --shift 3 > /dev/null || echo "command failed"
if python main.py crack-caesar --text "KH00R ZRU0G" --report; then echo "analysis saved"; fi
```

15. One-page command card

```
SETUP
  cd crypto_cipher_analyzer && python main.py                interactive menu

CAESAR
  python main.py caesar-encrypt --text "HELLO WORLD" --shift 3
  python main.py caesar-decrypt --text "KH00R ZRU0G" --shift 3

VIGENERE
  python main.py vigenere-encrypt --text "ATTACKATDAWN" --key LEMON
  python main.py vigenere-decrypt --text "LXF0PVEFRNHR" --key LEMON

ANALYSIS (add --report to save the result)
  python main.py frequency      --text "KH00R ZRU0G"
  python main.py crack-caesar   --text "KH00R ZRU0G" [--top 5]
  python main.py ic             --text "..." [--max-key-length 12]
  python main.py kasiski        --text "..." [--min-length 3 --max-length 5]
  python main.py analyze-vigenere --text "..." [--key-length N]
  python main.py security       [--cipher caesar|vigenere]

FILES
  python main.py file-caesar    --input FILE [--output FILE] --shift N [--decrypt] [--overwrite]
  python main.py file-vigenere --input FILE [--output FILE] --key WORD [--decrypt] [--overwrite]

TESTS
  python -m unittest discover tests

GLOBAL      python main.py --help | --version | --no-color <COMMAND> ...
```

Full documentation, worked examples and the completion status report are in `crypto_cipher_analyzer/docs/PROJECT_GUIDE.pdf`.